IFT6135 - Assignment 2 Report

Guillaume Genois, 20248507

March 20, 2026

Problem 2: Training Sequential Models

Question 1: Learning curves

Figures 1 to 8 show the training loss, validation loss, and validation accuracy over training iterations for all 8 experiments. Each figure corresponds to one experiment configuration trained on the Yelp Polarity sentiment analysis task. The plots are shown alongside the analysis in Questions 4 to 7.

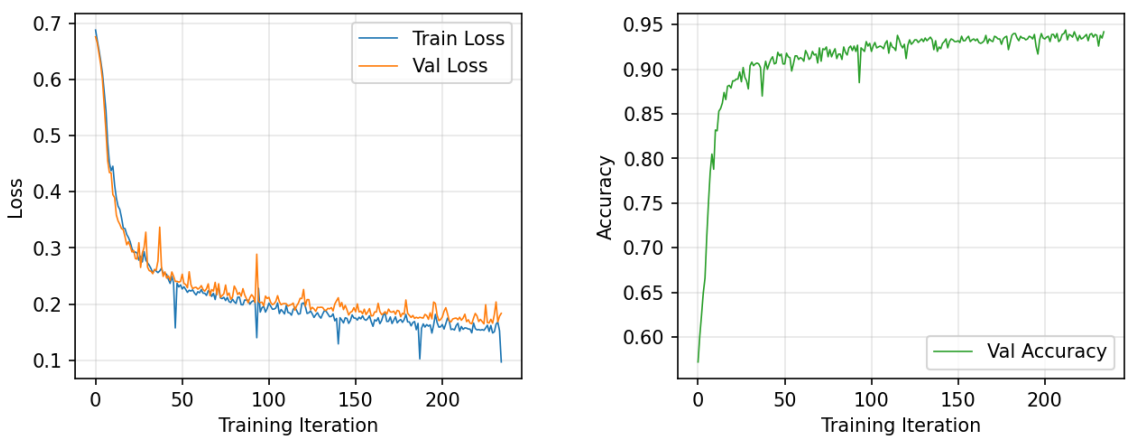

Figure 1: Experiment **1**: GRU, no dropout, encoder only.

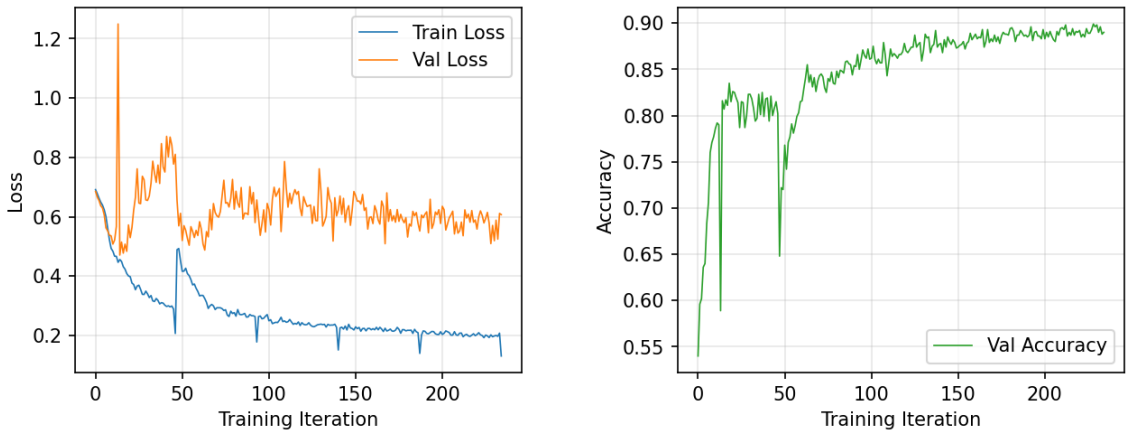

Figure 2: Experiment **2**: GRU, dropout, encoder only.

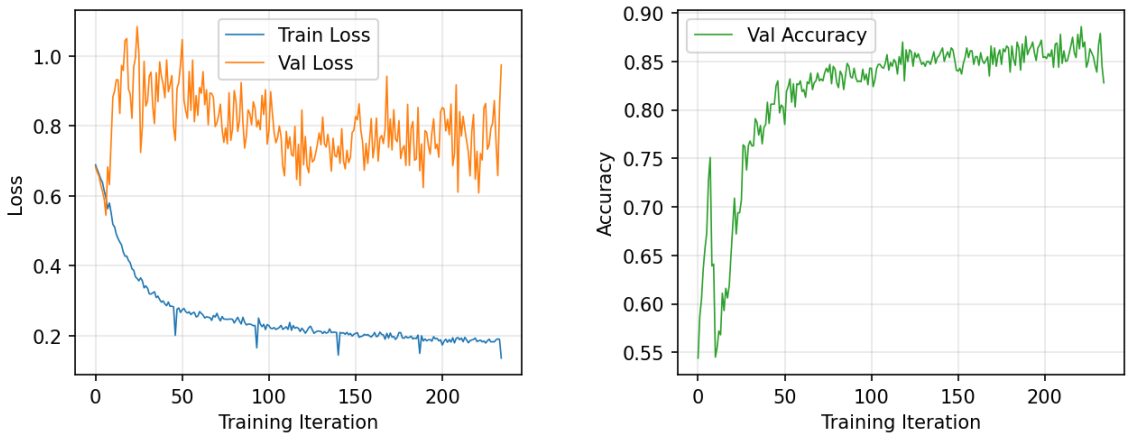

Figure 3: Experiment **3**: GRU, dropout, encoder-decoder, no attention.

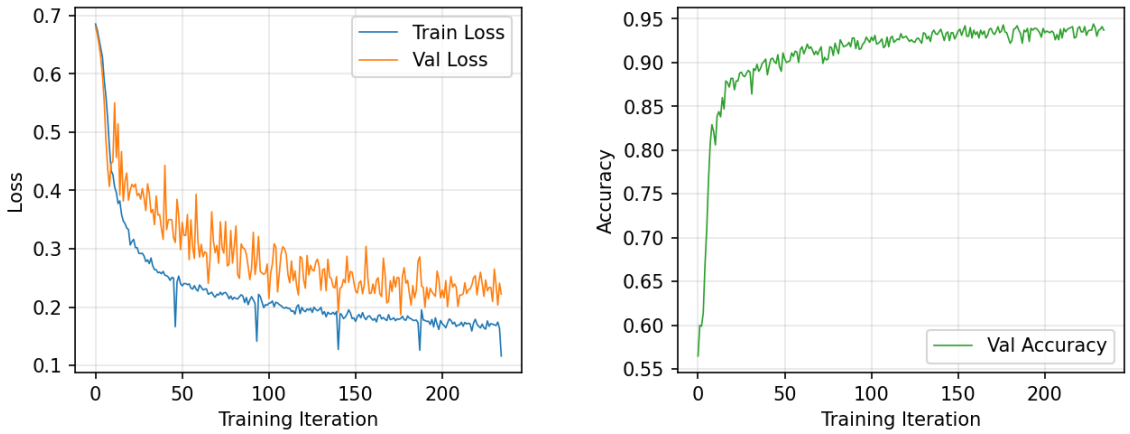

Figure 4: Experiment **4**: GRU, dropout, encoder-decoder, with soft attention.

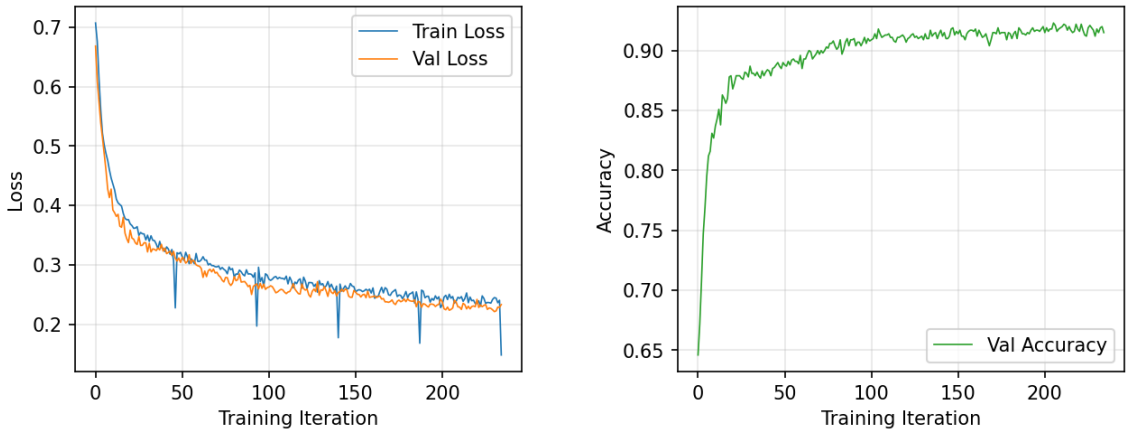

Figure 5: Experiment **5**: Transformer, 2 layers, prenorm.

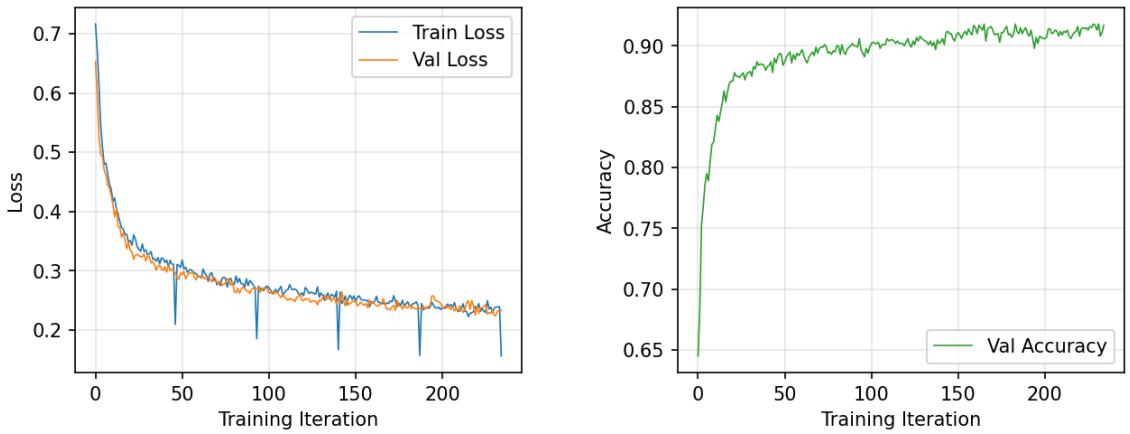

Figure 6: Experiment **6**: Transformer, 4 layers, prenorm.

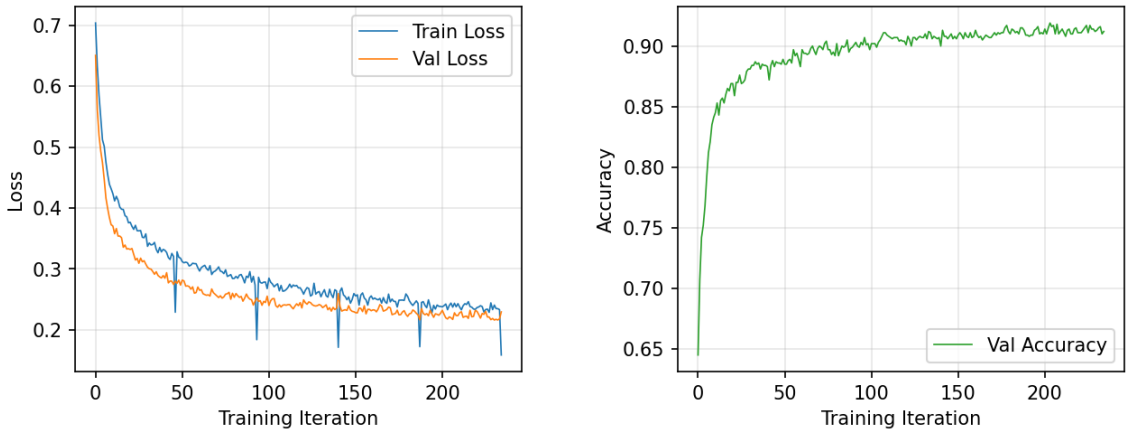

Figure 7: Experiment **7**: Transformer, 2 layers, postnorm.

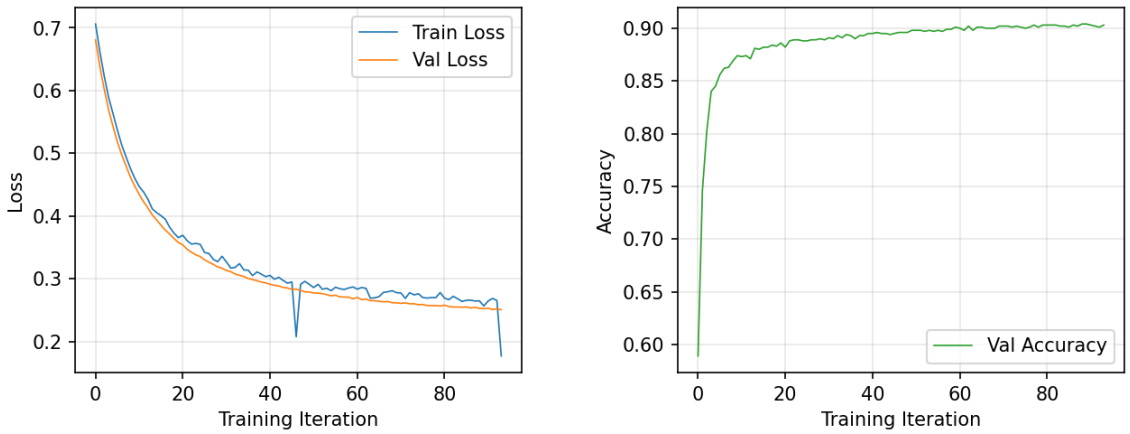

Figure 8: Experiment **8**: Pretrained BERT (2 epochs).

Question 2: Summary table

Table 1: Summary of results for all 8 experiments. Val Acc is the best validation accuracy across all training iterations. Train Loss is the final epoch training loss. Train and Eval columns are total training wall-clock time and per-epoch evaluation time. Best validation accuracy is in **bold**.

Exp	Architecture	Params	Val Acc	Train Loss	Train (min)	Eval (s)
1	GRU, no dropout, enc only	8.6M	**0.944**	0.159	15.4	0.18
2	GRU, dropout, enc only	8.6M	0.899	0.204	15.5	0.20
3	GRU, dropout, enc-dec	9.0M	0.886	0.187	20.7	0.22
4	GRU, dropout, enc-dec, attn	9.1M	**0.944**	0.172	23.1	0.23
5	Transformer 2L prenorm	8.7M	0.923	0.242	40.4	0.34
6	Transformer 4L prenorm	9.5M	0.918	0.237	75.1	0.56
7	Transformer 2L postnorm	8.7M	0.919	0.238	40.5	0.34
8	Pretrained BERT (2 ep.)	1.5K*	0.904	0.276	173.0	8.09

*Only the classification head parameters (1,538) are fine-tuned; the full BERT model has ∼110M parameters.

Question 3: Wall-clock time vs. generalization

If we care most about **wall-clock time**, experiments **1** or **2** (GRU encoder-only) seem like the best choices at about 15.4 to 15.5 minutes of total training. Between the two, experiment **1** also gets the highest validation accuracy (0.944), so it appears to be good for both speed and performance. These models are likely fast because they only do a single forward pass through the sequence with no decoder.

If we care most about **generalization**, experiments **1** and **4** are tied at 0.944 best validation accuracy. Experiment **4** (GRU encoder-decoder with attention) could potentially be a more robust choice since it uses dropout, which may help generalize better to new data.

Question 4: Impact of dropout

Comparing experiment **1** (no dropout, Figure 1) and experiment **2** (with dropout, Figure 2), it seems like dropout hurt the performance quite a lot, lowering the best validation accuracy from 0.944 to 0.899. The training loss for experiment **2** stays higher (0.204 vs 0.159), which is expected since dropout prevents the model from fitting the training data as tightly.

Looking at the validation loss, experiment **2** has much higher and noisier values than experiment **1**. This suggests that the dropout rate may be too strong for this setup, potentially regularizing the model too much and stopping it from learning good representations. Without dropout, the GRU encoder-only model does not seem to overfit badly as the validation accuracy keeps going up during training and the gap between train and val loss stays small. So in this case, it looks like dropout is not helpful and may actually make things worse.

Question 5: Encoder vs. Encoder-Decoder

Comparing experiment **2** (encoder-only, Figure 2) and experiment **3** (encoder-decoder without attention, Figure 3), the encoder-decoder seems to do slightly worse than the encoder-only model (best val acc 0.886 vs 0.899). Training time also goes up from 15.5 to 20.7 minutes.

That said, the difference is small (about 1.3 percentage points), so the two models are fairly close in practice. The encoder-decoder is slightly worse here probably because the decoder only gets a single fixed-size hidden vector from the encoder, which cannot keep all the important information from the full input. This creates an information bottleneck that can hurt performance. However, in theory, the encoder-decoder could be beneficial for tasks that require more complex reasoning over the input, since the decoder adds an extra processing step that can refine the representation before making a prediction. In our case, sentiment classification seems to be simple enough that the encoder alone is sufficient, and the extra decoder mostly adds training difficulty without much benefit.

Question 6: Impact of soft-attention

Comparing experiment **3** (no attention, Figure 3) and experiment **4** (with soft attention, Figure 4), adding soft attention seems to give a big improvement of about 5.8 percentage points in validation accuracy (0.886 to 0.944).

It looks like the attention mechanism helps fix the bottleneck problem as instead of squeezing the whole input into one hidden vector, the decoder can now look at all the encoder hidden states at each step. This likely lets the model focus on the most relevant parts of the input for its decision. The improvement brings the encoder-decoder with attention to the same level as the best model overall (experiment **1**). The validation loss also appears much lower and more stable. Training time only goes up a bit from 20.7 to 23.1 minutes, so attention seems to be very worth it.

Question 7: Transformer experiments

In experiments **5** (2-layer prenorm, Figure 5), **6** (4-layer prenorm, Figure 6), **7** (2-layer postnorm, Figure 7), and **8** (pretrained BERT, Figure 8), the Transformer models get decent results but not as good as the best GRU models (0.944).

This is a bit surprising given how popular Transformers are in NLP today. But there are a few possible reasons that could explain it:

- The Transformers here are very small (2 to 4 layers) compared to real models like BERT-base (12 layers) or GPT. Transformers seem to work better at larger scales.
- Sentiment analysis on Yelp is a fairly simple binary classification task. It seems like GRUs with attention can already handle this well without needing multi-head self-attention.
- With only 5 epochs of training (2 for BERT), the models may not have had enough time to fully learn. BERT gets 0.904 with just 2 epochs of fine-tuning and would likely improve with more epochs or better hyperparameter tuning.

Also, going from 2 layers (experiment **5**) to 4 layers (experiment **6**) does not seem to help and actually gives slightly worse results. This could suggest that at this small scale, adding more layers does not pay off and might even make optimization harder.

Question 8: GPU memory usage

Table 2: GPU memory usage for each experiment (MB). Values reported are the peak memory allocation during training, which we use as a proxy for steady-state usage since memory stays roughly constant after the first few iterations.

Exp.	Architecture	GPU Memory (MB)
1	GRU, no dropout, enc only	4,500
2	GRU, dropout, enc only	4,532
3	GRU, dropout, enc-dec	4,539
4	GRU, dropout, enc-dec, attn	4,941
5	Transformer 2L prenorm	5,626
6	Transformer 4L prenorm	10,395
7	Transformer 2L postnorm	5,427
8	Pretrained BERT	4,697

The GRU models (experiments **1** to **4**) use the least memory (4,500 to 4,941 MB). This is likely because RNNs process the sequence one step at a time and only need to keep the current hidden state in memory. Adding the attention mechanism in experiment **4** seems to increase memory a bit, probably because we need to store all the encoder hidden states for the attention computation.

The 2-layer Transformers (experiments **5** and **7**) use around 5,400 to 5,600 MB, which is more than the GRUs. This is probably because self-attention needs to compute and store $T \times T$ attention matrices, where T is the sequence length, so memory grows quadratically with sequence length.

Experiment **6** (4-layer Transformer) uses almost twice the memory at 10,395 MB compared to the 2-layer versions. Each extra layer adds its own attention matrices and intermediate values that need to be stored for backpropagation, which would explain this increase.

Experiment **8** (BERT) uses 4,697 MB, which is not that high. This is likely because only the small classification head is being fine-tuned, so the pretrained BERT weights are mostly frozen and there is no need to store gradients for all 110M parameters.

Question 9: Learning curve analysis

In experiment 1 (Figure 1), the train and val loss curves stay close together and both decrease smoothly throughout training. The val accuracy rises steadily up to about 0.94. There is no clear sign of overfitting here since the val loss keeps going down along with the train loss.

In experiment 2 (Figure 2), the train loss goes down smoothly, but the val loss is much higher (around 0.6) and stays noisy throughout training. There is a large gap between train and val loss, which suggests that dropout may be causing a mismatch between training and evaluation behavior. The val accuracy also has a visible drop around iteration 50 (near an epoch boundary) before recovering slowly. This seems to be the clearest case of overfitting in the GRU experiments, as the val loss does not really decrease after the first few iterations even though the train loss keeps going down.

In experiment 3 (Figure 3), the val loss is very noisy and stays high (between 0.6 and 1.0) for most of training. It sometimes spikes above 1.0. The val accuracy is also noisy and fluctuates a lot, especially in the first half of training. This looks like the most unstable experiment overall.

In experiment 4 (Figure 4), adding attention makes things much more stable compared to experiment 3. The val loss is lower and less noisy, and the val accuracy climbs more smoothly to around 0.94. There is still a visible gap between train and val loss, but both curves trend downward.

For the Transformers (experiments 5, 6, and 7 in Figures 5 to 7), the train and val loss curves stay very close to each other and decrease smoothly. The val accuracy rises steadily to around 0.91 to 0.92. These seem to be the most stable experiments with almost no noise in the curves. However, the train and val loss converge to similar values (around 0.23), which could suggest that the models still have room to learn if trained for longer.

In experiment 8 (Figure 8), BERT is only trained for 2 epochs so there are fewer data points. The train and val loss curves are very smooth and close together. The val accuracy reaches about 0.90 quickly and seems like it could still improve with more training.

Overall, the GRU models without dropout or with attention (experiments 1 and 4) seem to achieve the best accuracy, while the Transformer models seem to be the most stable during training. The models that showed the most instability are experiments 2 and 3, where dropout combined with the encoder-only or encoder-decoder setup led to noisy validation curves.

As for what a practitioner could do to improve things, here are a few ideas based on what we saw in class.

- To prevent overfitting, one could tune the dropout rate more carefully, since regularizers trade increased bias for reduced variance and an effective regularizer should reduce variance without overly increasing bias. Here, the dropout seems too aggressive and increases bias too much. Other options include L2 regularization (weight decay) or early stopping, which as we saw in class acts similarly to L2 regularization by restricting the optimization to a small volume of parameter space near the initial weights.

- To prevent underfitting, for small Transformers, training for more epochs could help since the loss curves suggest the models have not fully converged yet. As discussed in the self-attention lecture, the original Transformer paper showed that bigger models tend to perform better, so increasing model size could also help. Using learning rate warmup, as done in BERT training, is another common strategy to help Transformers converge better.

- To reduce instability, adding attention to encoder-decoder models (as in experiment 4) seems to help a lot compared to experiment 3, since it removes the information bottleneck. Gradient

clipping can also help with training instabilities by rescaling the gradient when its norm exceeds a threshold, which prevents the exploding gradient problem. For Transformers, using pre-norm instead of post-norm is another way to stabilize training, since it avoids putting layer normalization on the main residual signal path in longer runs.

Question 10: Cross Attention vs. Self Attention

An attention mechanism is a part of a neural network that lets the model focus on different parts of the input when making its output. It works by computing a weighted sum of a set of value vectors, where the weights come from how well a query vector matches a set of key vectors. This way, the model can pick out the most relevant information instead of having to compress everything into a single fixed-size vector. The weights are usually computed with a scaled dot-product or a small learned network, followed by a softmax to get a probability distribution over the input positions.

Self-attention is when the queries, keys, and values all come from the same sequence. Each position looks at every other position, which lets the model capture relationships between any two tokens no matter how far apart they are. Unlike RNNs, self-attention has no recurrence so all positions can be processed in parallel. In the Transformer, multi-head attention runs several attention heads in parallel, where each head can focus on different aspects of the input, and the results are concatenated and projected with a learned matrix W^O. Cross-attention, which is what we implemented as soft attention in the GRU encoder-decoder, is when the queries come from one sequence (the decoder hidden state) and the keys and values come from a different sequence (the encoder outputs). The Transformer decoder uses both masked self-attention on its own outputs and cross-attention (called encoder-decoder attention) to look at the encoder representations. The main difference between the two is where the queries and keys/values come from. In self-attention they come from the same place, which allows reasoning within a single sequence. In cross-attention they come from different places, which allows the decoder to pick up relevant information from the encoder at each step.